

Die Java-API kennenlernen – Teil 1

ArrayList, Boolesche Ausdrücke und Packages

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 1. Dezember 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Die Java-API kennenlernen

 Plan für diese Woche

 ArrayList

 Boolesche Ausdrücke

 Pair Programming

 Packages

 Erforschen der API

 Textbasiertes Interface

 Spaß

📖 Die Java-API kennenlernen

🔗 Verbindungen herstellen

- 👤 Tauschen Sie sich mit einer Nachbar:in aus!
- 🔗 Was ist der Unterschied zwischen einer ArrayList und einem Array?
- ↔ Welche Booleschen Ausdrücke kennen Sie? Welche haben Sie schon verwendet?
- 📦 Wie hängen Package-Name, Klassenname und der vollständige Name einer Klasse zusammen?
- 🙋 Welche Fragen haben Sie zu diesem Kapitel?

6 Die Java-API kennenlernen

Die Java-Bibliothek verwenden



Java wird mit Hunderten vorgefertigter Klassen ausgeliefert. Sie müssen das Rad nicht neu erfinden, wenn Sie wissen, wie Sie das Benötigte in der Java-Bibliothek, der sogenannten **Java-API**, finden können. *Sie haben Besseres zu tun.* Wenn Sie Code schreiben, reicht es völlig, wenn Sie sich *nur auf die Teile* konzentrieren, die bei Ihrer Anwendung besonders sind. Kennen Sie diese Programmierer, die jeden Tag um Punkt 5 nach Hause gehen und am nächsten Tag nicht vor 10 wieder auftauchen? **Sie benutzen die Java-API.** Und ungefähr acht Seiten weiter werden auch Sie das tun. Die Java-Kernbibliothek (Core Library) ist ein gigantischer Haufen Klassen, die nur darauf warten, dass Sie sie als Bausteine für Ihre eigenen Programme aus größtenteils vorgefertigtem Code verwenden. Die Java-Fertiggerichte, die wir in diesem Buch einsetzen, bestehen aus Code, den Sie nicht von Grund auf neu schreiben müssen. Eingeben müssen Sie ihn trotzdem. Die Java-API ist voller Code, den Sie nicht einmal *eingeben* müssen. Sie müssen nur lernen, wie man ihn benutzt.

ArrayList

Es klingt wie Magie...

🔗🔗 ArrayList

- 🪄 Es mag wie Zauberei klingen – so etwas gibt es wirklich.
- 🦸 Die Klasse ArrayList bietet eine Art Array mit Superkräften.
- ✂ Entfernt man Elemente, „schrumpft“ sie automatisch.
 - 😄 Sie brauchen sich also nicht um „leere Becher im Spindsystem“ zu kümmern.
- 👤 Eine Instanz der Klasse ArrayList ist ein „vollwertiges“ Objekt mit vielen nützlichen Methoden.



Ach, gäbe es doch nur ein Array, das **schrumpft**, wenn man etwas daraus **entfernt**. Und eins, über das man nicht iterieren muss, um die einzelnen Elemente zu überprüfen, eines, das man einfach **fragen kann**, ob es das **Gesuchte enthält**, ohne genau zu wissen, an welchem Index es sich befindet. Das wäre wirklich wunderbar, aber leider ist es doch nur ein Traum ...

„Okay... So eine ArrayList, die kann doch nicht nur schrumpfen, sondern auch wachsen, oder?“

Noel – neugierige Student:in im 1. Semester



Array mit Superkräften

🔗🔗 ArrayList

- `add(E e)`: Hängt das angegebene Element an das Ende der Liste.
- `remove(int index)`: Entfernt das Element an der angegebenen Position.
- `remove(Object o)`: Entfernt das erste Vorkommen des angegebenen Elements.
- `contains(Object o)`: Gibt `true` zurück, wenn die Liste das angegebene Element enthält.
- `isEmpty()`: Gibt `true` zurück, wenn die Liste kein Element enthält.
- `indexOf(Object o)`: Gibt entweder den ersten Index des Elements oder `-1` zurück.
- `size()`: Gibt die Anzahl der Elemente in der Liste zurück.
- `get(int index)`: Gibt das Element an der angegebenen Position zurück.



🔗 ArrayList

💬 Nachbarschaftskonferenz

- 👤 Tauschen Sie sich mit einer Nachbar:in aus!
- ! Vervollständigen Sie den Array-Code, indem Sie sich den ArrayList-Code ansehen.
- ? Wie müsste der Code aussehen, wenn es sich um ein reguläres Array handeln würde?
- ! Schauen Sie nicht in die Lösung, geben Sie einfach Ihr Bestes!



ArrayList

Nachbarschaftskonferenz

-  Tauschen Sie sich mit einer Nachbar:in aus!
- !** Vervollständigen Sie den Array-Code, indem Sie sich den ArrayList-Code ansehen.
- ?** Wie müsste der Code aussehen, wenn es sich um ein reguläres Array handeln würde?
- !** Schauen Sie nicht in die Lösung, geben Sie einfach Ihr Bestes!

ArrayList	Reguläres Array
<code>ArrayList<String> myList = new ArrayList<String>();</code>	<code>String [] myList = new String[2];</code>
<code>String a = "whoohoo";</code>	<code>String a = >>whoohoo<<;</code>
<code>myList.add(a);</code>	
<code>String b = "Frog";</code>	<code>String b = >>Frog<<;</code>
<code>myList.add(b);</code>	
<code>int theSize = myList.size();</code>	
<code>String str = myList.get(1);</code>	
<code>myList.remove(1);</code>	
<code>boolean isIn = myList.contains(b);</code>	 Seite 134

! Quiz

🔗 ArrayList

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



`ArrayList` ist eine Klasse, die von der Java-API zur Verfügung gestellt wird.

Richtig!

Um etwas zu einer `ArrayList` hinzuzufügen,
verwenden Sie die Methode `add()`.

Richtig!

Um etwas aus einer `ArrayList` zu entfernen, verwenden Sie `delete()`.

... verwenden Sie `remove()`.

Um herauszufinden, wo sich etwas in einer `ArrayList` befindet (und ob es überhaupt vorhanden ist), verwenden Sie die Methode `indexOf()`.

Richtig!

Um herauszufinden, ob eine `ArrayList` leer ist, verwenden Sie `hasContent()`.

... verwenden Sie `isEmpty()`.

Um die Größe (Anzahl der Elemente) einer `ArrayList` zu ermitteln, verwenden Sie die Methode `size()`.

Richtig!

Um die Länge (Anzahl der Elemente) eines normalen, einfachen Arrays zu bestimmen, verwenden Sie die Methode `getLength()`.

... verwenden Sie die Variable `length`.

Eine `ArrayList` passt ihre Größe dynamisch an die aktuellen Größenbedürfnisse an. Sie wächst, wenn Elemente hinzugefügt werden, und schrumpft, wenn Elemente entfernt werden.

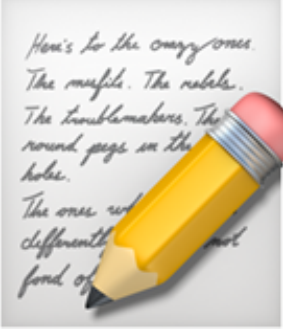
Richtig!

Um den Typ des Arrays der Liste zu deklarieren, verwenden Sie einen Typparameter, also einen Typnamen in spitzen Klammern.
Zum Beispiel: `ArrayList<Person>` bedeutet, dass die `ArrayList` nur Objekte vom Typ `Person` enthalten darf (oder Unterklassen von `Person`, wie Sie in den folgenden Wochen lernen werden).

Richtig!

Obwohl eine `ArrayList` Objekte und keine elementaren Typen aufnehmen kann, „wrappt“ (und „unwrappt“) sie einen elementaren Typ automatisch, wenn Sie ihn in eine `ArrayList` stecken (oder wieder herausholen), und packt statt des elementaren Typs dann dieses Objekt in die `ArrayList`.

Richtig!

 Welcher **eine** Gedanke zu
„  ArrayList“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren

Boolesche Ausdrücke

Here's to the crazy ones. The misfits. The troublemakers. The round pegs in the square holes. The ones who see things differently. They are not fond of your rules. And they have no respect for status quo. You can quote them, disagree with them, ignore them, or vilify them. About the only thing you can't do is

Sie haben dafür 2 Minuten Zeit.

! ? Quiz

↔ Boolesche Ausdrücke

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Mit dem Ausdruck `price >= 300 && price <= 400` können Sie testen, ob der Preis im Bereich zwischen 300 und 400 (z. B. Euro) liegt.

Richtig!

Mit dem Ausdruck `name("Christian") || name("Julia")` können Sie testen, ob der Name entweder Christian oder Julia ist.

Richtig!

Mit dem Ausdruck `name != "Paul"` können Sie testen, ob der Name ungleich Paul ist.

Fast: Verwenden Sie für den Vergleich von Objekten wie Strings `!obj1.equals(obj2)`.

Die Operatoren && („und“) und || („oder“) werden als Kurzschluss-Operatoren bezeichnet, weil sie die Auswertung eines Ausdrucks vorzeitig abbrechen können, sobald das Ergebnis feststeht.

Richtig!

Beim Operator `&&` wird die rechte Seite nur überprüft, wenn die linke Seite wahr ist. Ist die linke Seite falsch, wird die Auswertung abgebrochen, da der Ausdruck dann insgesamt falsch ist.

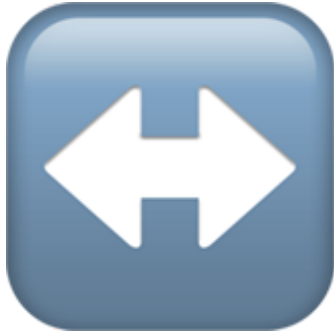
Beispiel: Bei `false && . . .` wird die rechte Seite überhaupt nicht überprüft, da das Ergebnis immer `false` bleibt.

Richtig!

Beim `||`-Operator ist der Ausdruck wahr, wenn mindestens eine Seite wahr ist. Wird die linke Seite als wahr erkannt, bricht die Auswertung ab, da das Ergebnis dann bereits feststeht.

Beispiel: Bei `false || ...` wird die rechte Seite nicht überprüft, da das Ergebnis immer `false` ist.

Fast: `true || irgendwas` → „irgendwas“ wird nicht evaluiert

 Welcher **eine** Gedanke zu
„ Boolesche Ausdrücke“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 **ArrayList und Boolesche Ausdrücke**

🤝 **Pair Programming**

- 🎯 **Entwickeln Sie ein Programm, das eine zufällige Anzahl von Zufallszahlen generiert und je nach deren Eigenschaften unterschiedliche Meldungen ausgibt.**
- ✅ **Legen Sie zunächst zufällig die Anzahl (zwischen 1 und 10) der zu erzeugenden Zufallszahlen fest.**
- ✅ **Füllen Sie dann eine ArrayList mit der entsprechenden Anzahl von Zufallszahlen zwischen 1 und 20.**



🎓 ArrayList und Boolesche Ausdrücke

🤝 Pair Programming

- 🎯 Entwickeln Sie ein Programm, das eine zufällige Anzahl von Zufallszahlen generiert und je nach deren Eigenschaften unterschiedliche Meldungen ausgibt.
- ✅ Legen Sie zunächst zufällig die Anzahl (zwischen 1 und 10) der zu erzeugenden Zufallszahlen fest.
- ✅ Füllen Sie dann eine ArrayList mit der entsprechenden Anzahl von Zufallszahlen zwischen 1 und 20.

- ✅ Prüfen Sie für jede Zufallszahl, ob sie entweder ... oder ...
- ✅ zwischen 1 und 10 liegt und gerade ist
- ✅ zwischen 11 und 20 liegt und ungerade ist.
- 🧙 Der Ausdruck $\text{number \% 2} = 0$ prüft, ob eine Zahl gerade ist.
- 🚩 Es soll eine entsprechende Ausgabe erfolgen.

```
1 import java.util.ArrayList;
2 import java.util.Random;
3
4 public class RandomNumberChecker {
5     public static void main(String[] args) {
6         Random random = new Random();
7
8         int numberOfRandomNumbers = random.nextInt(10) + 1;
9         System.out.println("Number of random numbers to generate: " + numberOfRandomNumbers);
10
11         ArrayList<Integer> randomNumbers = new ArrayList<>();
12
13         for (int i = 0; i < numberOfRandomNumbers; i++) {
14             randomNumbers.add(random.nextInt(20) + 1);
15         }
16
17         System.out.println();
18         System.out.print("Generated " + randomNumbers.size() + " random numbers: ");
19         System.out.println(randomNumbers);
20
21         System.out.println();
22         System.out.println("Checking each number:");
23         System.out.println();
24
25         for (int number : randomNumbers) {
26             if ((number >= 1 && number <= 10 && number % 2 == 0)) {
27                 System.out.println("- " + number + " is between 1 and 10 and is even.");
28             } else if ((number >= 11 && number <= 20 && number % 2 != 0)) {
29                 System.out.println("- " + number + " is between 11 and 20 and is odd.");
30             } else {
31                 System.out.println("- " + number + " does not meet any of the specified conditions.");
32             }
33         }
34     }
35 }
36
```

STDIN

Input for the program (Optional)

Output:

Number of random numbers to generate: 7

Generated 7 random numbers: [5, 2, 6, 13, 9, 6, 7]

Checking each number:

- 5 does not meet any of the specified conditions.
- 2 is between 1 and 10 and is even.
- 6 is between 1 and 10 and is even.
- 13 is between 11 and 20 and is odd.
- 9 does not meet any of the specified conditions.
- 6 is between 1 and 10 and is even.
- 7 does not meet any of the specified conditions.

 Welcher **eine** Gedanke zu
„ ArrayList und Boolesche
Ausdrücke“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren



Packages

**Jetzt Seiten 154
bis 157
wiederholen!**



Sie haben dafür 5 Minuten Zeit.

Organisationssystem

Packages

 Packages helfen im Wesentlichen dabei, **Ihr Programm zu organisieren.**

 Statt einer unübersichtlichen Ansammlung von Klassen werden sie in Packages für verschiedene Arten von Funktionalität gruppiert.

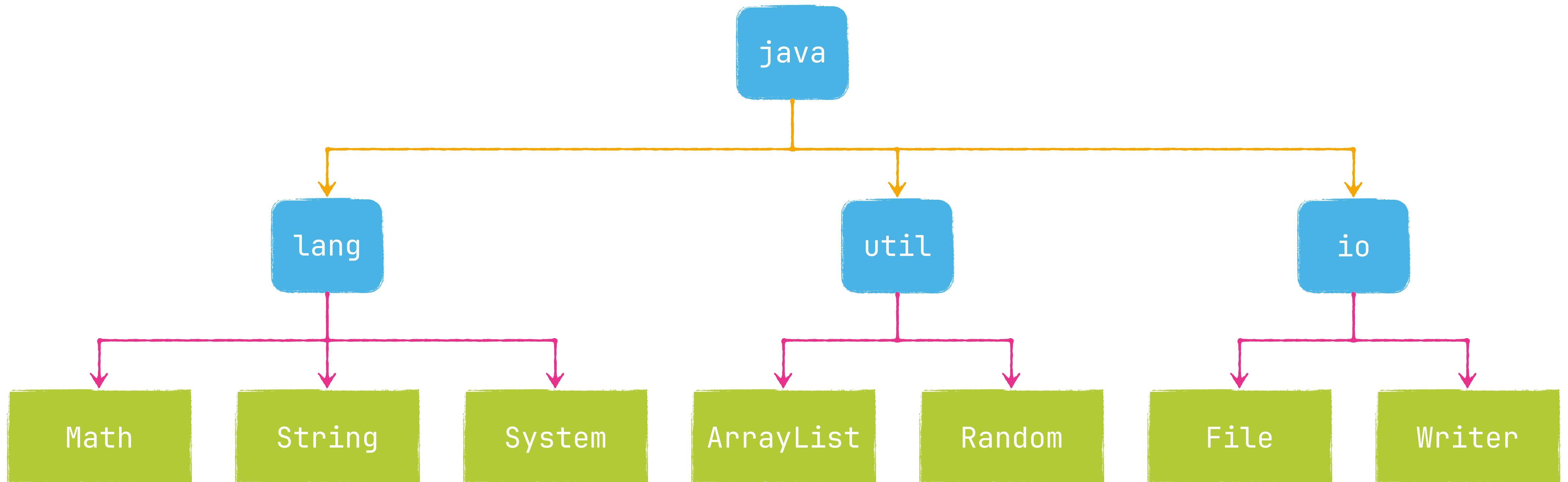
 Wie User Interface, Anwendungslogik, Helferklassen, und so weiter.

 Spoiler: Niedrige Kopplung – Hohe Kohäsion



Organisation von Klassen in Paketen

 Packages



„Mmh... Deswegen muss ich Klassen wie `System` und `String` nicht explizit importieren, andere wie `Random` und `ArrayList` aber sehr wohl!“

Merle – aufmerksame Student:in im 1. Semester



Sichtbarkeitsbereich

Packages

- 👁️ Packages definieren einen **namensbasierten Sichtbarkeitsbereich (Scope)**, der Kollisionen verhindert.
- ⚠️ Wenn Sie eine Klasse mit dem Namen `ArrayList` haben und jemand anderes (einschließlich der Java-API) hat auch eine Klasse mit diesem Namen, müssen Sie irgendwie der JVM mitteilen, welche Klasse mit dem Namen `ArrayList` verwendet werden soll.



„Ha! `de.sesqa.pr1.ArrayList` ist eine andere Klasse als `java.util.ArrayList`.“

Gerit – aufgeweckte Student:in im 1. Semester



! ? Quiz

Packages

-  Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
-  Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
-  Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Klassen werden zu Packages (dt. Pakete)
gruppiert.

Richtig!

Eine Klasse hat einen vollständigen Namen, der sich aus dem Package-Namen und dem Klassennamen zusammensetzt. Die Klasse `ArrayList` heißt eigentlich `java.util.ArrayList`.

Richtig!

Für die Verwendung einer Klasse aus einem anderen Package als `java.lang` müssen Sie Java den vollständigen Namen der JVM mitteilen.

Richtig!

Sie können entweder eine `include`-Anweisung am Anfang Ihres Quellcodes verwenden oder den vollständigen Namen an jeder Stelle eintippen, an der Sie die Klasse in Ihrem Code verwenden.

... eine `import`-Anweisung am Anfang...

 Welcher **eine** Gedanke zu
„ Packages“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Die Java-API kennenlernen

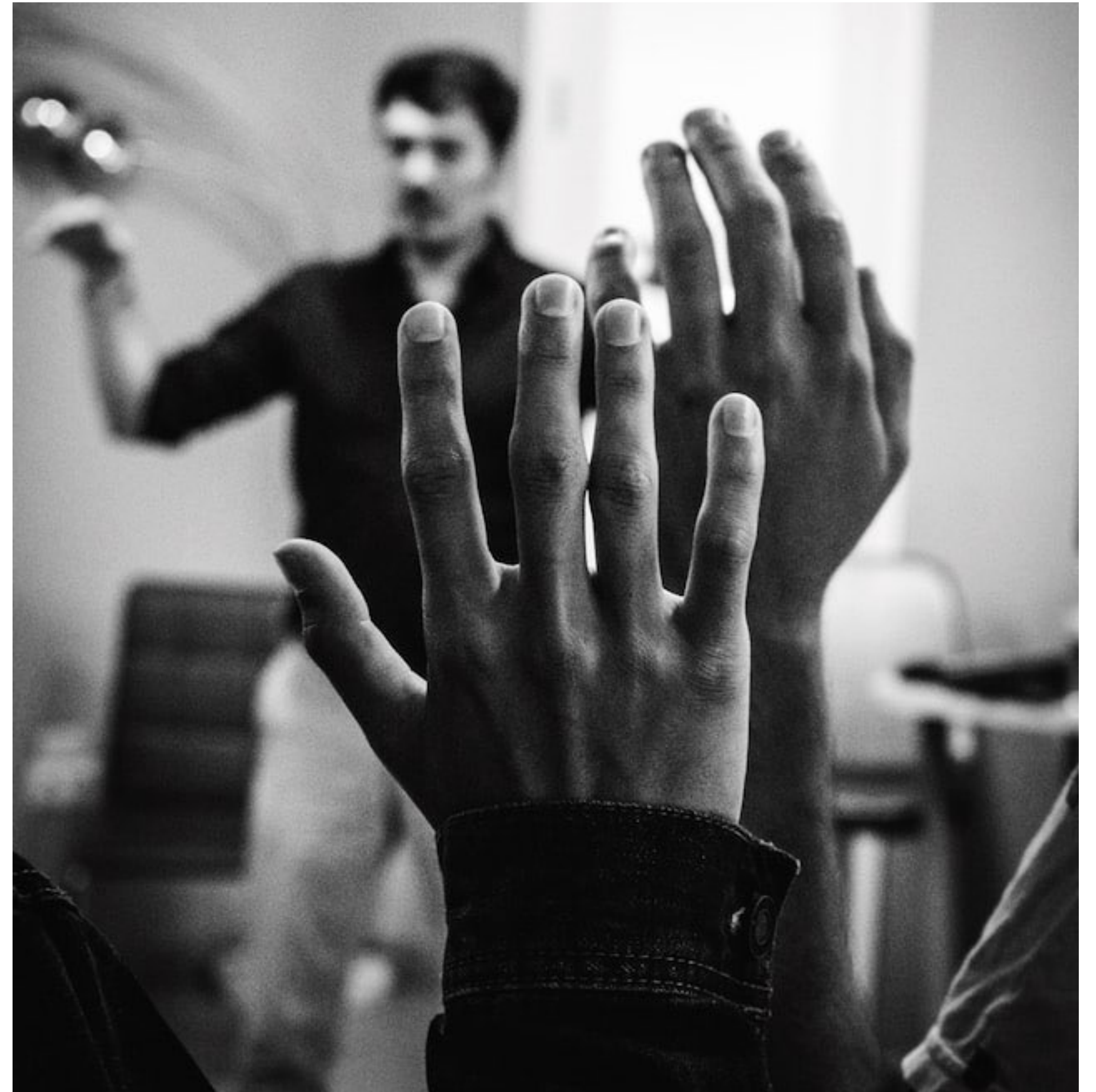
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Die Java-API kennenlernen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Die Java-API kennenlernen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

